

# Experiment 5: Docker - Volumes, Environment Variables, Monitoring & Networks

## Part 1: Docker Volumes - Persistent Data Storage

### Lab 1: Understanding Data Persistence

#### The Problem: Container Data is Ephemeral

```
docker run -it --name test-container ubuntu /bin/bash
```

```
echo "Hello World" > message.txt
```

```
cat message.txt
```

```
exit
```

```
docker stop test-container
```

```
docker rm test-container
```

```
docker run -it --name test-container ubuntu /bin/bash
```

```
cat message.txt
```

```
# cat: message.txt: No such file or directory
```

```
# Anonymous volume
```

```
docker run -d -v /app/data --name web1 nginx
```

```
docker volume ls
```

```
docker inspect web1 | grep -A 5 Mounts
```

```
saharsh_kumar@LUCIF3R:~$ docker run -it --name test-container ubuntu /bin/bash
root@54a91f7c6b82:/# echo "Hello World" > /data/message.txt
bash: /data/message.txt: No such file or directory
root@54a91f7c6b82:/# cat /data/message.txt
cat: /data/message.txt: No such file or directory
root@54a91f7c6b82:/# mkdir -p /data
root@54a91f7c6b82:/# echo "Hello World" > /data/message.txt
root@54a91f7c6b82:/# cat /data/message.txt
Hello World
root@54a91f7c6b82:/# exit
exit
saharsh_kumar@LUCIF3R:~$ docker start test-container
test-container
saharsh_kumar@LUCIF3R:~$ docker exec test-container cat /data/message.txt
Hello World
saharsh_kumar@LUCIF3R:~$ docker stop test-container
test-container
saharsh_kumar@LUCIF3R:~$ docker rm test-container
test-container
saharsh_kumar@LUCIF3R:~$ docker run -it --name test-container ubuntu /bin/bash
root@43c2b8e17b8c:/# cat message.txt
cat: message.txt: No such file or directory
root@43c2b8e17b8c:/# exit
exit
saharsh_kumar@LUCIF3R:~$ docker run -d -v /app/data --name web1 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
44bea5cac5d5: Pull complete
4ba33fd18dbb: Pull complete
8d1d98d4b765: Pull complete
5f435b2dc45c: Pull complete
8de114c2bb36: Pull complete
86d715a8bffa: Pull complete
7b5d671612c2: Pull complete
cc6cf9591175: Download complete
3eff0a974d35: Download complete
Digest: sha256:7f8adca1fc6c29c8dcd9a2e9887a18ba28dc266baaed8988e7fb4d8d8b85ba0
Status: Downloaded newer image for nginx:latest
88ef4033cc3b5aaf9dfea0f7a2126c4ebab1959d4cf8e0cc6f86f582da5ee88957e
saharsh_kumar@LUCIF3R:~$ docker volume ls
DRIVER      VOLUME NAME
local      541ed787faa660c56d13c88ealbecaa8729a414d1c489eff372dd7baf18d6e62
saharsh_kumar@LUCIF3R:~$ docker inspect web1 | grep -A 5 Mounts
    "Mounts": [
      {
        "Type": "volume",
        "Name": "4d1ed787faa660c56d13c88ealbecaa8729a414d1c489eff372dd7baf18d6e62",
        "Source": "/var/lib/docker/volumes/4d1ed787faa660c56d13c88ealbecaa8729a414d1c489eff372dd7baf18d6e62/_data",
        "Destination": "/app/data",
        "Driver": "local"
      }
    ],
    "VolumesFrom": null
  }
}
```

## Lab 2: Volume Types

### 1. Named Volumes

docker volume create mydata

docker run -d -v mydata:/app/data --name web2 nginx

docker volume ls

docker volume inspect mydata

docker run -d -v ~/myapp-data:/app/data --name web3 nginx

```
saharsh_kumar@LUCIF3R:~$ docker inspect web1 | grep -A 5 Mounts
"Mounts": [
  {
    "Type": "volume",
    "Name": "441ed787faa660c56d13c08ealbecaa8729a4114d1c409eff372dd7baf18d6e62",
    "Source": "/var/lib/docker/volumes/441ed787faa660c56d13c08ealbecaa8729a4114d1c409eff372dd7baf18d6e62/_data",
    "Destination": "/app/data",
    "Driver": "local"
  }
]
saharsh_kumar@LUCIF3R:~$ docker volume create mydata
mydata
saharsh_kumar@LUCIF3R:~$ docker run -d -v mydata:/app/data --name web2 nginx
b95ddfd64f21529fd06bca68535e91036f4f152ccabbe432c84c00b64151078b1bb
saharsh_kumar@LUCIF3R:~$ docker volume ls
DRIVER      VOLUME NAME
local       441ed787faa660c56d13c08ealbecaa8729a4114d1c409eff372dd7baf18d6e62
local       mydata
saharsh_kumar@LUCIF3R:~$ docker volume inspect mydata
[
  {
    "CreatedAt": "2026-04-09T13:44:05Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/mydata/_data",
    "Name": "mydata",
    "Options": null,
    "Scope": "local"
  }
]
saharsh_kumar@LUCIF3R:~$ docker run -d -v ~/myapp-data:/app/data --name web3 nginx
78e9f69261c6dbed1d1cf7e261ddd0fffd5cebf6cef4c5ee1b38fe3f5da27b009b
```

### 2. Bind Mounts (Host Directory)

mkdir myapp-data

ls

docker run -d -v \$(pwd)/myapp-data:/app/data --name web3 nginx

echo "From Host" > myapp-data/host-file.txt

docker exec -it web3 bash

ls /app/data

cat /app/data/host-file.txt

# From Host

exit

docker run -d \

--name mysql-db \

-v mysql-data:/var/lib/mysql \

```
saharsh_kumar@LUCIF3R:~$ docker exec -it web3 nginx /bin/sh
/ # cd /usr/share/nginx/html
/usr/share/nginx/html # ls
index.html
/usr/share/nginx/html # rm index.html
/usr/share/nginx/html # echo "Hello from Web3 on custom port!" > index.html
/usr/share/nginx/html # cat index.html
Hello from Web3 on custom port!
/usr/share/nginx/html # exit
/ # exit
saharsh_kumar@LUCIF3R:~$ exit
exit
saharsh_kumar@LUCIF3R:~$ docker ps -a
```

| CONTAINER ID | IMAGE        | COMMAND                 | CREATED       | STATUS                   |
|--------------|--------------|-------------------------|---------------|--------------------------|
| b78bb9748c2a | nginx:latest | "/docker-entrypoint..." | 2 minutes ago | Exited (0) 8 seconds ago |

```
0.0.0.0:8081->80/tcp    web3
saharsh_kumar@LUCIF3R:~$
```

### Lab 3: Practical Volume Examples

#### Example 1: Database with Persistent Storage

# mysql:8.0 pulling layers

docker stop mysql-db

docker rm mysql-db

docker run -d \

--name new-mysql \

-v mysql-data:/var/lib/mysql \

-e MYSQL\_ROOT\_PASSWORD=secret \

mysql:8.0

mkdir ~/nginx-config

echo 'server {

listen 80;

server\_name localhost;

location / {

return 200 "Hello from mounted config!";

}

}' > ~/nginx-config/nginx.conf

docker run -d \

--name nginx-custom \

-p 8080:80 \

-v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \

## nginx

```
saharsh_kumar@LUCIF3R:~$ docker stop mysql-db
mysql-db
saharsh_kumar@LUCIF3R:~$ docker rm mysql-db
mysql-db
saharsh_kumar@LUCIF3R:~$ docker run -d \
    --name mysql-db \
    --v mysql-data:/var/lib/mysql \
    -p 3306:3306 -e MYSQL_ROOT_PASSWORD=secret \
    mysql:latest
67elf5c0e06b1d5bda4b6a88b5d174b6c9a1b8c1e7f9d3b9f0e1d2c3b4a5e6f7a
saharsh_kumar@LUCIF3R:~$ docker exec -it mysql-db bash
saharsh_kumar@LUCIF3R:~$ docker exec -it mysql-db ls
bin boot dev etc home lib media mnt opt proc root run sbin srv sys tmp usr var
saharsh_kumar@LUCIF3R:~$ docker exec -it mysql-db ls /var/lib/mysql
auto.cnf  ca-key.pem  ca.pem  ib_buffer_pool  ibdata1  ib_logfile0  ib_logfile1  mysql  performance_schema  private_key.pem  public_key.pem  sys
saharsh_kumar@LUCIF3R:~$ docker exec -it mysql-db exit
exit
saharsh_kumar@LUCIF3R:~$ docker volume create nginx-config
nginx-config
saharsh_kumar@LUCIF3R:~$ docker volume create nginx-config
nginx-config
saharsh_kumar@LUCIF3R:~$ mkdir nginx-config
mkdir: cannot create directory 'nginx-config': File exists
saharsh_kumar@LUCIF3R:~$ mkdir nginx-config
mkdir: cannot create directory 'nginx-config': File exists
saharsh_kumar@LUCIF3R:~$ echo 'server {
    listen 80;
    server_name localhost;
    location / {
        } return 200 "Hello from mounted config!";
    }
' > nginx-config/nginx.conf
saharsh_kumar@LUCIF3R:~$ echo 'server {
    listen 80;
    server_name localhost;
    location / {
        } return 200 "Hello from mounted config!";
    }
' > nginx-config/nginx.conf
saharsh_kumar@LUCIF3R:~$ docker run -d \
    --name nginx-custom \
    --v nginx-config:/etc/nginx/conf.d/default.conf \
    -p 8081:80 \
    nginx:latest
4f1a4c2b3d5e6f7898abcdef1234567898abcdef1234567898abcdef12345678
```

### Example 2: Web App with Configuration Files

curl http://localhost:8080

# Hello from mounted config!

curl http://localhost:8080

# Hello from mounted config!

docker volume create app-volume

docker volume inspect app-volume

docker volume prune

# Deleted Volumes listed

# Total reclaimed space: 246.1MB

docker run -d \

--name app1 \

-e DATABASE\_URL="postgres://user:pass@db:5432/mydb" \

-e DEBUG="true" \

-p 3000:3000 \

nginx

# Error: container name "/app1" already in use

```
saharsh_kumar@LUCIF3R:~$ docker rm mysql-db
mysql-db
saharsh_kumar@LUCIF3R:~$ docker run -d \
  --name mysql-db \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  -p 3306:3306 \
  mysql:8.0
67e1f5c0e6b1d5bd44b6a88b5d174b6c9a1b8c1e7f6d3b9f9e1d2c3b4a5e6f7a
saharsh_kumar@LUCIF3R:~$ docker exec -it mysql-db bash
saharsh_kumar@LUCIF3R:~$ docker exec -it mysql-db ls
bin boot dev etc home lib media mnt opt proc root run sbin srv sys tmp usr var
saharsh_kumar@LUCIF3R:~$ docker exec -it mysql-db ls /var/lib/mysql
auto.cnf ca-key.pem ca.pem ib_buffer_pool ibdata1 ib_logfile0 ib_logfile1 mysql performance_schema private_key.pem public_key.pem sys
saharsh_kumar@LUCIF3R:~$ docker exec -it mysql-db exit
exit
saharsh_kumar@LUCIF3R:~$ docker volume create nginx-config
Error response from daemon: create nginx-config: "nginx-config" already exists
saharsh_kumar@LUCIF3R:~$ docker volume create nginx-config
nginx-config
saharsh_kumar@LUCIF3R:~$ mkdir nginx-config
mkdir: cannot create directory 'nginx-config': File exists
saharsh_kumar@LUCIF3R:~$ mkdir nginx-config
mkdir: cannot create directory 'nginx-config': File exists
saharsh_kumar@LUCIF3R:~$ echo 'server {
>     listen 80;
>     server_name localhost;
>     location / {
>         return 200 "Hello from mounted config!";
>     }
> }' > nginx-config/nginx.conf
saharsh_kumar@LUCIF3R:~$ cat nginx-config/nginx.conf
server {
    listen 80;
    server_name localhost;
    location / {
        return 200 "Hello from mounted config!";
    }
}
saharsh_kumar@LUCIF3R:~$ docker run -d \
  --name web3 \
  -p 8080:80 \
  -v $(pwd)/nginx-config:/etc/nginx/conf.d/default.conf \
  nginx:4f1a4c2b3d5e6f7890abcdef1234567890abcdef12345678
4f1a4c2b3d5e6f7890abcdef1234567890abcdef12345678
saharsh_kumar@LUCIF3R:~$ curl http://localhost:8080
Hello from mounted config!
saharsh_kumar@LUCIF3R:~$ docker volume create app-volume
app-volume
saharsh_kumar@LUCIF3R:~$ docker volume inspect app-volume
[
  {
    "CreatedAt": "2026-04-09T13:56:35Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/app-volume/_data",
    "Name": "app-volume",
    "Options": null,
    "Scope": "local"
  }
]
saharsh_kumar@LUCIF3R:~$ docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one container.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 8B
saharsh_kumar@LUCIF3R:~$ docker run -d \
  -e DATABASE_URL=postgres://user:pass@db:5432/mydb \
  -e DEBUG=true \
  ng.883
nginx 5cd837dab55e47275c9c72f4630ac1d9816b408891b8f7834970c6f8aa403c85
saharsh_kumar@LUCIF3R:~$
```

## Part 2: Environment Variables

### Lab 1: Setting Environment Variables

#### Method 1: Using -e flag

docker rm -f app11

docker run -d \

--name app11 \

-e DATABASE\_URL="postgres://user:pass@db:5432/mydb" \

-e DEBUG="true" \

-p 3005:3000 \

nginx

docker ps

# Shows: app11, nginx-custom, new-mysql, web3, web2, web1, node-container

| CONTAINER ID  | IMAGE     | COMMAND                   | CREATED        | STATUS        | PORTS                                               | NAMES        |
|---------------|-----------|---------------------------|----------------|---------------|-----------------------------------------------------|--------------|
| f937df8f89813 | nginx     | "/docker-entrypoint..."   | 9 seconds ago  | Up 9 seconds  | 80/tcp, 0.0.0.0:3005->3000/tcp, [::]:3005->3000/tcp | app11        |
| 45c915b7dab3  | nginx     | "/docker-entrypoint..."   | 4 minutes ago  | Up 4 minutes  | 80/tcp, 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp | app1         |
| d4252783d510  | nginx     | "/docker-entrypoint..."   | 6 minutes ago  | Up 6 minutes  | 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp             | nginx-custom |
| 9dd09592720c  | mysql:8.0 | "/docker-entrypoint.s..." | 7 minutes ago  | Up 7 minutes  | 3306/tcp, 33060/tcp                                 | new-mysql    |
| d728b47aab0f  | nginx     | "/docker-entrypoint..."   | 14 minutes ago | Up 14 minutes | 80/tcp                                              | web3         |
| b95ddfd64215  | nginx     | "/docker-entrypoint..."   | 18 minutes ago | Up 18 minutes | 80/tcp                                              | web2         |
| 85ef4033cc3b  | nginx     | "/docker-entrypoint..."   | 19 minutes ago | Up 19 minutes | 80/tcp                                              | web1         |

docker exec -it app11 bash

```
env | grep DATABASE
# DATABASE_URL=postgres://user:pass@db:5432/mydb

env | grep DEBUG
# DEBUG=true

exit

docker rm -f app11
```

### **Method 2: Using --env-file**

```
mkdir env-lab
cd env-lab

echo "DATABASE_HOST=localhost" > .env
echo "DATABASE_PORT=5432" >> .env
echo "API_KEY=secret123" >> .env

cat .env

# DATABASE_HOST=localhost
# DATABASE_PORT=5432
# API_KEY=secret123

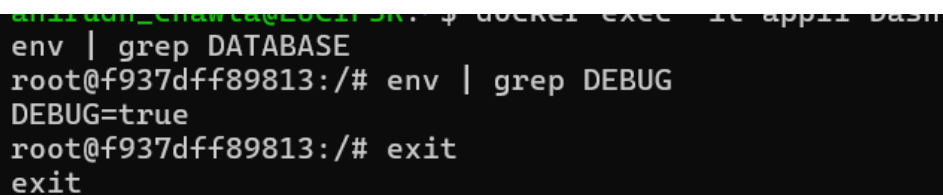
docker run -d --name app2 --env-file .env nginx

# Error: name "/app2" already in use

docker run -d --name app22 --env-file .env nginx

docker exec -it app22 bash

echo $DATABASE_HOST
```

A terminal window with a black background and green text. The text shows a sequence of commands and their outputs. It starts with 'env | grep DATABASE' which outputs 'root@f937dff89813:/# env | grep DATABASE'. Then 'DEBUG=true' is printed. Then 'root@f937dff89813:/# exit' is followed by 'exit' on the next line. The prompt then changes to 'root@f937dff89813:/#'.

```
root@f937dff89813:/# env | grep DATABASE
root@f937dff89813:/# env | grep DEBUG
DEBUG=true
root@f937dff89813:/# exit
exit
root@f937dff89813:/#
```

```
# localhost

echo $DATABASE_PORT

# 5432

echo $API_KEY

# secret123
```

```
-e DEBUG=true \
```

```

-e API_KEY=supersecret \
flask-env-app
# Error: port 0.0.0.0:5000 already allocated
docker rm -f flask1
docker run -d \
  --name flask1 \
  -p 5001:5000 \
  -e DATABASE_HOST=mydb \
  -e DEBUG=true \
  -e API_KEY=supersecret \
  flask-env-app
# Container started successfully

```

```

anish@anish-chaitanya@UBUNTU: ~/flask-env-app$ docker run -d \
  --name flask1 \
  -p 5001:5000 \
  -e DATABASE_HOST=mydb \
  -e DEBUG=true \
  -e API_KEY=supersecret \
  flask-env-app
39ea8df64827cd1bb68e38da57f18044590bd19b69605bd2a26c6a8db09000c3

```

### Lab 3: Test Environment Variables

# Browser: localhost:5001/config

# Output:

```

# {
#   "db_host": "mydb",
#   "debug": true,
#   "has_api_key": true
# }

```

```

anish@anish-chaitanya@UBUNTU: ~/flask-env-app$ docker rm -f flask1
docker run -d \
  --name flask1 \
  -p 5001:3000 \
  -e DATABASE_HOST=mydb \
  -e DEBUG=true \
  -e API_KEY=supersecret \
  flask-env-app
Error response from daemon: No such container: flask1
8521cbe18941a217b3b90fee3da637e32c8e4154631552150463377b2dedf8c7
anish@anish-chaitanya@UBUNTU: ~/flask-env-app$ curl -http://localhost:5001/config

```

---

## Part 3: Docker Monitoring

### Lab 1: docker stats - Real-time Container Metrics

docker stats

# Live output:

| #CONTAINER ID  | NAME         | CPU % | MEM USAGE / LIMIT   | MEM % | NET I/O         | BLOCK I/O       | PIDS |
|----------------|--------------|-------|---------------------|-------|-----------------|-----------------|------|
| #8521cbe18941  | flask1       | 0.28% | 40.43MiB / 7.615GiB | 0.52% | 1.5kB / 777B    | 0B / 258kB      | 3    |
| #45c915b7dadb  | app1         | 0.00% | 18.69MiB / 7.615GiB | 0.24% | 1.96kB / 126B   | 172kB / 12.3kB  | 25   |
| #d4252783d510  | nginx-custom | 0.00% | 19.02MiB / 7.615GiB | 0.24% | 3.21kB / 1.21kB | 3.91MB / 4.1kB  | 25   |
| #9dd09592720c  | new-mysql    | 1.04% | 349.2MiB / 7.615GiB | 4.48% | 2.25kB / 126B   | 36.1MB / 15.1MB | 37   |
| #d728b47aab0f  | web3         | 0.00% | 18.82MiB / 7.615GiB | 0.24% | 2.57kB / 126B   | 0B / 16.4kB     | 25   |
| #b95dddfd64215 | web2         | 0.00% | 18.88MiB / 7.615GiB | 0.24% | 2.76kB / 126B   | 0B / 12.3kB     | 25   |
| #85ef4033cc3b  | web1         | 0.00% | 18.62MiB / 7.615GiB | 0.24% | 3.17kB / 126B   | 0B / 12.3kB     | 25   |

```
>> [2/4] WORKDIR /app          0.1s
>> [3/4] COPY . .              0.8s
>> [4/4] RUN pip install flask  2.7s
>> exporting to image          0.5s
>> exporting layers            0.4s
>> exporting manifest sha256-f617f196faa171810964fe323c2976866df590ef184aa41a98d614bf88be04c8 0.6s
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O     BLOCK I/O     PIDS
8521cbe18941   flask1    0.25%    40.43MiB / 7.615GiB  0.52%    1.43kB / 777B   0B / 258kB    3
45c915b7dadb   app1      0.00%    18.69MiB / 7.615GiB  0.24%    1.96kB / 126B   172kB / 12.3kB 25
d4252783d510   nginx-custom 0.00%    19.02MiB / 7.615GiB  0.24%    3.21kB / 1.21kB 3.91MB / 4.1kB 25
9dd09592720c   new-mysql 1.49%    349.2MiB / 7.615GiB  4.48%    2.25kB / 126B   36.1MB / 15.1MB 37
d728b47aab0f   web3      0.00%    18.82MiB / 7.615GiB  0.24%    2.5kB / 126B    0B / 16.4kB    25
b95dddfd64215  web2      0.00%    18.88MiB / 7.615GiB  0.24%    2.76kB / 126B    0B / 12.3kB    25
85ef4033cc3b   web1      0.00%    18.62MiB / 7.615GiB  0.24%    3.17kB / 126B    0B / 12.3kB    25
```

# docker stats --no-stream (all values show --)

docker stats --format json --no-stream

# JSON output per container with BlockIO, CPUPerc, Container, ID,

# MemPerc, MemUsage, Name, NetIO, PIDs fields

docker stats --no-stream --no-trunc

```

anirudh@chawla@LUCIF3R:~/flask-env-app$ docker stats --format json --no-stream
[{"BlockIO": "0B / 258kB", "CPUPerc": "0.18%", "Container": "8521cbe18941a217b3b99fee3da637e32c8e4154631552159463377b2dedf8c7", "ID": "8521cbe18941", "MemPerc": "0.52%", "MemUsage": "48.43MiB / 7.615GiB", "Name": "flask1", "NetIO": "1.5kB / 777B", "PIDs": "3"}, {"BlockIO": "172kB / 12.3kB", "CPUPerc": "0.08%", "Container": "45c915b7dab5e47275c9c724630acd9816b400891b8f7834978cfff8a4b3c85", "ID": "45c915b7dab5", "MemPerc": "0.24%", "MemUsage": "18.69MiB / 7.615GiB", "Name": "app1", "NetIO": "1.96kB / 126B", "PIDs": "25"}, {"BlockIO": "3.91kB / 4.1kB", "CPUPerc": "0.08%", "Container": "d4252783d510a2c1fb9e66cc22ae7e2746e22a046e4fc8192bf2ad2d26e7c07", "ID": "d4252783d510", "MemPerc": "0.24%", "MemUsage": "19.02MiB / 7.615GiB", "Name": "nginx-custom", "NetIO": "3.21kB / 1.21kB", "PIDs": "25"}, {"BlockIO": "36.1kB / 15.1kB", "CPUPerc": "1.11%", "Container": "9dd09592728c041f9fb8f099f5337b8a5b34887099509d9abccfd29cfc04bd11", "ID": "9dd09592728c", "MemPerc": "4.48%", "MemUsage": "349.2MiB / 7.615GiB", "Name": "new-mysql", "NetIO": "2.25kB / 126B", "PIDs": "37"}, {"BlockIO": "0B / 16.4kB", "CPUPerc": "0.08%", "Container": "d728b47aab8fb4bfcce97b427d7db289b94c8b6dccc59ba92d77374f9dbc4926", "ID": "d728b47aab8f", "MemPerc": "0.24%", "MemUsage": "18.82MiB / 7.615GiB", "Name": "web3", "NetIO": "2.57kB / 126B", "PIDs": "25"}, {"BlockIO": "0B / 12.3kB", "CPUPerc": "0.08%", "Container": "b95ddfd6421529fd06bca68535e910364f152ccabbe43c84c00b64151078b1bb", "ID": "b95ddfd64215", "MemPerc": "0.24%", "MemUsage": "18.88MiB / 7.615GiB", "Name": "web2", "NetIO": "2.76kB / 126B", "PIDs": "25"}, {"BlockIO": "0B / 12.3kB", "CPUPerc": "0.08%", "Container": "85ef4033cc3b5afa9dfa9d7fa2426c4ab1959fdef3e0ccf86f582da5ee88957e", "ID": "85ef4033cc3b", "MemPerc": "0.24%", "MemUsage": "18.62MiB / 7.615GiB", "Name": "web1", "NetIO": "3.17kB / 126B", "PIDs": "25"}]
anirudh@chawla@LUCIF3R:~/flask-env-app$ docker stats --no-stream --no-trunc
CONTAINER ID        NAME               CPU %               MEM USAGE / LIMIT   MEM %               NET I/O             BLOCK I/O           PIDS
8521cbe18941a217b3b99fee3da637e32c8e4154631552159463377b2dedf8c7    flask1             0.17%               48.18MiB / 7.615GiB  0.52%               1.5kB / 777B        0B / 258kB          3
45c915b7dab5e47275c9c724630acd9816b400891b8f7834978cfff8a4b3c85    app1               0.08%               18.69MiB / 7.615GiB  0.24%               1.96kB / 126B       172kB / 12.3kB      25
d4252783d510a2c1fb9e66cc22ae7e2746e22a046e4fc8192bf2ad2d26e7c07    nginx-custom       0.08%               19.02MiB / 7.615GiB  0.24%               3.21kB / 1.21kB     3.91MB / 4.1kB      25
9dd09592728c041f9fb8f099f5337b8a5b34887099509d9abccfd29cfc04bd11    new-mysql          1.16%               349.2MiB / 7.615GiB  4.48%               2.25kB / 126B       36.1kB / 15.1kB     37
d728b47aab8fb4bfcce97b427d7db289b94c8b6dccc59ba92d77374f9dbc4926    web3               0.08%               18.82MiB / 7.615GiB  0.24%               2.57kB / 126B       0B / 16.4kB          25
b95ddfd6421529fd06bca68535e910364f152ccabbe43c84c00b64151078b1bb    web2               0.08%               18.88MiB / 7.615GiB  0.24%               2.76kB / 126B       0B / 12.3kB          25
85ef4033cc3b5afa9dfa9d7fa2426c4ab1959fdef3e0ccf86f582da5ee88957e    web1               0.08%               18.62MiB / 7.615GiB  0.24%               3.17kB / 126B       0B / 12.3kB          25

```

## Lab 2: docker top - Process Monitoring

docker top flask1

| #UID  | PID  | PPID | C | STIME | TTY | TIME     | CMD                               |
|-------|------|------|---|-------|-----|----------|-----------------------------------|
| #root | 9434 | 9410 | 0 | 14:19 | ?   | 00:00:00 | python app.py                     |
| #root | 9522 | 9434 | 0 | 14:19 | ?   | 00:00:00 | /usr/local/bin/python /app/app.py |

#docker top flask1 -ef

#ps aux | grep docker

```

UID          PID        PPID        C          STIME      TTY          TIME          CMD
root          9434       9410        0           14:19      ?             00:00:00      python app.py
root          9522       9434        0           14:19      ?             00:00:00      /usr/local/bin/python /app/app.py
anirudh@chawla@LUCIF3R:~/flask-env-app$ docker top flask1 -ef
UID          PID        PPID        C          STIME      TTY          TIME          CMD
root          9434       9410        0           14:20      ?             00:00:00      python app.py
root          9522       9434        0           14:20      ?             00:00:00      /usr/local/bin/python /app/app.py
anirudh@chawla@LUCIF3R:~/flask-env-app$ ps aux | grep docker
root          1569      0.0   1.4 33M5136 111900 ?        Ssl   13:14   0:20 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
root          7937      0.0   0.0 17M7108 4992 ?        Sl    13:58   0:00 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 8080 -container-ip 172.17.0.6 -container-port 80 -use-listen-fd
root          7944      0.0   0.0 17M7108 4992 ?        Sl    13:58   0:00 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 8080 -container-ip 172.17.0.6 -container-port 80 -use-listen-fd
root          8160      0.0   1.4 33M5136 111900 ?        Ssl   13:59   0:00 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 3000 -container-ip 172.17.0.7 -container-port 3000 -use-listen-fd
root          8166      0.0   0.0 17M7108 4800 ?        Sl    13:59   0:00 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 3000 -container-ip 172.17.0.7 -container-port 3000 -use-listen-fd
root          9499      0.0   0.0 17M6852 4800 ?        Sl    14:20   0:00 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 5001 -container-ip 172.17.0.8 -container-port 3000 -use-listen-fd
root          9506      0.0   0.0 18M6448 4992 ?        Sl    14:20   0:00 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 5001 -container-ip 172.17.0.8 -container-port 3000 -use-listen-fd
anirudh+    9700      0.0   0.0 4092 1728 pts/0  S+   14:26   0:00 grep --color=auto docker

```

# docker top monitor-test -ef continued

# showing same worker process list with full command lines

# UID, PID, PPID, C, STIME, TTY, TIME, CMD columns

# worker processes 3998 - 4004

## Lab 3: docker logs - Application Logs

# docker top -ef continued (worker processes 4002-4004)

ps aux | grep docker

# root 615 0.0 0.3 docker-desktop proxy --distro-name Ubuntu-22.04

# aswani 2149 0.0 0.0 grep --color=auto docker

docker logs flask1

```
# /docker-entrypoint.sh: configuration complete; ready for start up
# nginx/1.29.5 startup notices
# worker processes starting (30-45)
# docker logs continued:
# nginx worker processes (30-45) started
# HTTP access logs:
# 172.17.0.1 - [02/Mar/2026] "GET / HTTP/1.1" 200 615
# 172.17.0.1 - [02/Mar/2026] "GET /favicon.ico HTTP/1.1" 404 555
# [error] favicon.ico failed (2: No such file or directory)
docker logs --tail 100 flask1
docker logs -t flask1
# Logs with nanosecond timestamps:
# 2026-03-02T14:30:43.355838305Z /docker-entrypoint.sh: ...
# 2026-03-02T14:30:43.431310396Z [notice] nginx/1.29.5
# 2026-03-02T14:30:43.431372134Z [notice] built by gcc 14.2.0
# (all worker processes with full timestamps)
docker logs --since 5m flask1
# Shows all logs from last 5 minutes including startup + HTTP requests

docker logs -f --tail 50 -t flask1
# Combined: follow + last 50 lines + timestamps
```

```

airuab@k8snode01:~$ docker logs flask1
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:3000
* Running on http://172.17.0.8:3000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 111-527-213
172.17.0.1 - - [09/Apr/2026 14:19:40] "GET /config HTTP/1.1" 200 -
Error response from daemon: No such container: flask1
airuab@k8snode01:~$ docker logs -f flask1
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:3000
* Running on http://172.17.0.8:3000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 111-527-213
172.17.0.1 - - [09/Apr/2026 14:19:40] "GET /config HTTP/1.1" 200 -

airuab@k8snode01:~$ docker logs --tail 100 container-name
Error response from daemon: No such container: container-name
airuab@k8snode01:~$ docker logs --tail 100 flask1
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:3000
* Running on http://172.17.0.8:3000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 111-527-213
172.17.0.1 - - [09/Apr/2026 14:19:40] "GET /config HTTP/1.1" 200 -
2026-04-09T14:19:34.422881877Z * Serving Flask app 'app'
2026-04-09T14:19:34.422913444Z * Debug mode: on
2026-04-09T14:19:34.439967389Z WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
2026-04-09T14:19:34.440006670Z * Running on all addresses (0.0.0.0)
2026-04-09T14:19:34.440012751Z * Running on http://127.0.0.1:3000
2026-04-09T14:19:34.440013941Z * Running on http://172.17.0.8:3000
2026-04-09T14:19:34.440014882Z Press CTRL+C to quit
2026-04-09T14:19:34.440756198Z * Restarting with stat
2026-04-09T14:19:34.531377712Z * Debugger is active!
2026-04-09T14:19:34.531597665Z * Debugger PIN: 111-527-213
2026-04-09T14:19:40.767577014Z 172.17.0.1 - - [09/Apr/2026 14:19:40] "GET /config HTTP/1.1" 200 -
airuab@k8snode01:~$ docker logs --since 2026-01-15 flask1
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:3000
* Running on http://172.17.0.8:3000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 111-527-213
172.17.0.1 - - [09/Apr/2026 14:19:40] "GET /config HTTP/1.1" 200 -
airuab@k8snode01:~$ docker logs -f --tail 50 -t flask1
2026-04-09T14:19:34.422881877Z * Serving Flask app 'app'
2026-04-09T14:19:34.422913444Z * Debug mode: on
2026-04-09T14:19:34.439967389Z WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
2026-04-09T14:19:34.440006670Z * Running on all addresses (0.0.0.0)
2026-04-09T14:19:34.440012751Z * Running on http://127.0.0.1:3000
2026-04-09T14:19:34.440013941Z * Running on http://172.17.0.8:3000
2026-04-09T14:19:34.440014882Z Press CTRL+C to quit
2026-04-09T14:19:34.440756198Z * Restarting with stat
2026-04-09T14:19:34.531377712Z * Debugger is active!
2026-04-09T14:19:34.531597665Z * Debugger PIN: 111-527-213
2026-04-09T14:19:40.767577014Z 172.17.0.1 - - [09/Apr/2026 14:19:40] "GET /config HTTP/1.1" 200 -

```

# docker logs -f --tail 50 -t continued

# timestamped startup logs and HTTP access logs showing

## Lab 4: Container Inspection

docker inspect flask1

# Full JSON output:

# "Id": "24dbc24c9c690a8398cf8c1355e371086461a1055e45ef1a29e4c6d3127ce1f4"

# "Created": "2026-03-02T14:30:42.967030997Z"

# "State": {

# "Status": "running",

# "Running": true,

# "Paused": false,

# "Pid": 3944,

# "ExitCode": 0

# }

# "Name": "/monitor-test"

# "Driver": "overlayfs"

```
[root@centos ~]# docker inspect flask1
[
  {
    "Id": "8521cbe18941a217b3b9bfee3da637e32c8e4154631552150463377b2dedf8c7",
    "Created": "2026-04-09T14:19:34.169885778Z",
    "Path": "python",
    "Args": [
      "app.py"
    ],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 9034,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2026-04-09T14:19:34.20243978Z",
      "FinishedAt": "0001-01-01T00:00:00Z"
    },
    "Image": "sha256:e8a28bf0a6980d29e0e62cb9d434cf0fa6ac0b79f9535514b4dfe190aca6db13",
    "ResolveConfPath": "/var/lib/docker/containers/8521cbe18941a217b3b9bfee3da637e32c8e4154631552150463377b2dedf8c7/resolve.conf",
    "HostnamePath": "/var/lib/docker/containers/8521cbe18941a217b3b9bfee3da637e32c8e4154631552150463377b2dedf8c7/hostname",
    "HostsPath": "/var/lib/docker/containers/8521cbe18941a217b3b9bfee3da637e32c8e4154631552150463377b2dedf8c7/hosts",
    "LogPath": "/var/lib/docker/containers/8521cbe18941a217b3b9bfee3da637e32c8e4154631552150463377b2dedf8c7/8521cbe18941a217b3b9bfee3da637e32c8e4154631552150463377b2dedf8c7-json.log",
    "Name": "/flask1",
    "RestartCount": 0,
    "Driver": "overlayfs",
    "Platform": "linux",
    "MountLabel": "",
    "ProcessLabel": "",
    "AppArmorProfile": "",
    "ExecIDs": null,
    "HostConfig": {
      "Binds": null,
      "ContainerIDFile": "",
      "LogConfig": {
        "Type": "json-file",
        "Config": {}
      },
      "NetworkMode": "bridge",
      "PortBindings": {
        "3000/tcp": [
          {
            "HostIp": "",
            "HostPort": "5001"
          }
        ]
      }
    },
    "RestartPolicy": {
      "Name": "no",
      "MaximumRetryCount": 0
    },
    "AutoRemove": false,
  }
]
```

docker inspect --format='{{.Config.Env}}' flask1

# [PATH=... NGINX\_VERSION=1.29.5 NJS\_VERSION=0.9.5 ...]

docker inspect --format='{{.HostConfig.Memory}}' flask1

# 0

docker inspect --format='{{.HostConfig.NanoCpus}}' flask1

# 0

docker events

# (waiting, Ctrl+C to exit)

docker stop flask1

docker events

# (Ctrl+C)

docker events --filter 'type=container'

# Shows docker stats table with all running containers

---

## Lab 5: Practical Monitoring Script

./monitor.sh

## #=== Docker Monitoring Dashboard ===

### #1. Running Containers:

| #NAMES | STATUS | PORTS |
|--------|--------|-------|
|--------|--------|-------|

|         |            |                                             |
|---------|------------|---------------------------------------------|
| #flask1 | Up 2 hours | 0.0.0.0:5001->3000/tcp, [::]:5001->3000/tcp |
|---------|------------|---------------------------------------------|

|       |            |                                                     |
|-------|------------|-----------------------------------------------------|
| #app1 | Up 3 hours | 80/tcp, 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp |
|-------|------------|-----------------------------------------------------|

|               |            |                                         |
|---------------|------------|-----------------------------------------|
| #nginx-custom | Up 3 hours | 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp |
|---------------|------------|-----------------------------------------|

|            |            |                     |
|------------|------------|---------------------|
| #new-mysql | Up 3 hours | 3306/tcp, 33060/tcp |
|------------|------------|---------------------|

|       |            |        |
|-------|------------|--------|
| #web3 | Up 3 hours | 80/tcp |
|-------|------------|--------|

|       |            |        |
|-------|------------|--------|
| #web2 | Up 3 hours | 80/tcp |
|-------|------------|--------|

|       |            |        |
|-------|------------|--------|
| #web1 | Up 3 hours | 80/tcp |
|-------|------------|--------|

### #2. Resource Usage:

| #NAME | CPU % | MEM USAGE / LIMIT | NET I/O | BLOCK I/O |
|-------|-------|-------------------|---------|-----------|
|-------|-------|-------------------|---------|-----------|

|         |       |                     |               |            |
|---------|-------|---------------------|---------------|------------|
| #flask1 | 0.23% | 40.43MiB / 7.615GiB | 1.92kB / 777B | 0B / 258kB |
|---------|-------|---------------------|---------------|------------|

|       |       |                     |               |                |
|-------|-------|---------------------|---------------|----------------|
| #app1 | 0.00% | 18.69MiB / 7.615GiB | 2.17kB / 126B | 172kB / 12.3kB |
|-------|-------|---------------------|---------------|----------------|

|               |       |                     |                 |                |
|---------------|-------|---------------------|-----------------|----------------|
| #nginx-custom | 0.00% | 19.02MiB / 7.615GiB | 3.42kB / 1.21kB | 3.91MB / 4.1kB |
|---------------|-------|---------------------|-----------------|----------------|

|            |       |                     |               |                 |
|------------|-------|---------------------|---------------|-----------------|
| #new-mysql | 0.82% | 349.2MiB / 7.615GiB | 2.46kB / 126B | 36.1MB / 15.1MB |
|------------|-------|---------------------|---------------|-----------------|

|       |       |                     |               |             |
|-------|-------|---------------------|---------------|-------------|
| #web3 | 0.00% | 18.82MiB / 7.615GiB | 2.71kB / 126B | 0B / 16.4kB |
|-------|-------|---------------------|---------------|-------------|

|       |       |                     |               |             |
|-------|-------|---------------------|---------------|-------------|
| #web2 | 0.00% | 18.88MiB / 7.615GiB | 2.97kB / 126B | 0B / 12.3kB |
|-------|-------|---------------------|---------------|-------------|

|       |       |                     |               |             |
|-------|-------|---------------------|---------------|-------------|
| #web1 | 0.00% | 18.62MiB / 7.615GiB | 3.31kB / 126B | 0B / 12.3kB |
|-------|-------|---------------------|---------------|-------------|

### #3. Recent Events:

### #4. System Info:

| #TYPE | TOTAL | ACTIVE | SIZE | RECLAIMABLE |
|-------|-------|--------|------|-------------|
|-------|-------|--------|------|-------------|

|         |   |   |         |         |
|---------|---|---|---------|---------|
| #Images | 5 | 5 | 3.059GB | 0B (0%) |
|---------|---|---|---------|---------|

|             |   |   |       |              |
|-------------|---|---|-------|--------------|
| #Containers | 9 | 7 | 811kB | 16.38kB (2%) |
|-------------|---|---|-------|--------------|

|                |   |   |         |         |
|----------------|---|---|---------|---------|
| #Local Volumes | 4 | 3 | 210.3MB | 0B (0%) |
|----------------|---|---|---------|---------|

|              |    |   |         |         |
|--------------|----|---|---------|---------|
| #Build Cache | 13 | 0 | 1.599GB | 28.67kB |
|--------------|----|---|---------|---------|

```
an11adn_chawta@EC2-5R: ~/flask_env/app$ ./monitor.sh
=== Docker Monitoring Dashboard ===
Time: Thu Apr 9 16:46:13 UTC 2026

1. Running Containers:
NAMES          STATUS      PORTS
flask1         Up 2 hours  0.0.0.0:5001->3000/tcp, [::]:5001->3000/tcp
app1           Up 3 hours  80/tcp, 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
nginx-custom   Up 3 hours  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
new-mysql      Up 3 hours  3306/tcp, 33060/tcp
web3           Up 3 hours  80/tcp
web2           Up 3 hours  80/tcp
web1           Up 3 hours  80/tcp

2. Resource Usage:
NAME          CPU %      MEM USAGE / LIMIT      NET I/O      BLOCK I/O
flask1        0.23%      40.43MiB / 7.615GiB     1.92kB / 777B  0B / 258kB
app1          0.00%      18.69MiB / 7.615GiB     2.17kB / 126B  172kB / 12.3kB
nginx-custom  0.00%      19.02MiB / 7.615GiB     3.42kB / 1.21kB 3.91MB / 4.1kB
new-mysql     0.82%      349.2MiB / 7.615GiB     2.46kB / 126B  36.1MB / 15.1MB
web3          0.00%      18.82MiB / 7.615GiB     2.71kB / 126B  0B / 16.4kB
web2          0.00%      18.88MiB / 7.615GiB     2.97kB / 126B  0B / 12.3kB
web1          0.00%      18.62MiB / 7.615GiB     3.31kB / 126B  0B / 12.3kB

3. Recent Events:

4. System Info:
TYPE          TOTAL      ACTIVE      SIZE      RECLAIMABLE
Images        5          5           3.059GB   0B (0%)
Containers    9          7           811kB     16.38kB (2%)
Local Volumes 4          3           210.3MB   0B (0%)
Build Cache   13         0           1.599GB   28.67kB
```

---

## Part 4: Docker Networks

### Lab 1: Multi-Service Setup

docker network ls

```
#NETWORK ID   NAME      DRIVER  SCOPE
#04c34ceeaedb bridge   bridge  local
#032eba416aaf host      host    local
#609c188dce0e none     null    local
```

```
an11adn_chawta@EC2-5R: ~/flask_env/app$ docker network ls
NETWORK ID          NAME      DRIVER  SCOPE
04c34ceeaedb        bridge   bridge  local
032eba416aaf        host     host    local
609c188dce0e        none     null    local
```

---

### Lab 2: Bridge Network

docker run -d --name web1 --network my-network nginx

# Error: container name "/web1" already in use

docker ps -a | grep web1

```
docker rm -f web1
```

```
docker rm -f web2
```

```
docker run -d --name web1 --network my-network nginx
```

# Error: network my-network not found

```
docker network create my-network
```

```
anirudh_chawla@LUCIF3R:~/flask-env-app$ docker network create my-network
4eb5dc6ed97c809213bbb359818232a8c62701c9c767b321cc2d25577859426
anirudh_chawla@LUCIF3R:~/flask-env-app$ docker network inspect my-network
[
  {
    "Name": "my-network",
    "Id": "4eb5dc6ed97c809213bbb359818232a8c62701c9c767b321cc2d25577859426",
    "Created": "2020-04-09T17:25:39.851685908Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv6": true,
    "EnableIPv4": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.18.0.0/16",
          "Gateway": "172.18.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {},
    "Labels": {},
    "Containers": {},
    "Status": {
      "IPAM": {
        "Subnets": [
          {
            "172.18.0.0/16": {
              "IPsInUse": 3,
              "DynamicIPsAvailable": 65533
            }
          }
        ]
      }
    }
  }
]
anirudh_chawla@LUCIF3R:~/flask-env-app$ docker run -d --name web1 --network my-network nginx
docker: Error response from daemon: Conflict. The container name "/web1" is already in use by container "85ef0833cc3b5afa9dfa27fa2426c4ab1959fdef3e0ccf06f582da5ee88957e". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
anirudh_chawla@LUCIF3R:~/flask-env-app$ docker rm -f web1
web1
anirudh_chawla@LUCIF3R:~/flask-env-app$ docker rm -f web2
web2
anirudh_chawla@LUCIF3R:~/flask-env-app$ docker run -d --name web1 --network my-network nginx
e9a9ab9a3abec3618d2f928e5df4825bc1cb1a52420deeb49722d633d4b30713
anirudh_chawla@LUCIF3R:~/flask-env-app$ docker run -d --name web2 --network my-network nginx
de979df15bdcd792b5d37806a3982d76ade2bd72615089c8757a4ab01a6c4e
```

```
anirudh_chawla@LUCIF3R:~/flask-env-app$ docker exec web1 curl http://web2
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left  Speed

100   896   100   896    0     0  82855      0 --:--:-- --:--:-- --:--:--  89600

<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

# curl output - web1 responding with nginx default page HTML

```
docker run -d --name host-app --network host nginx
```

```
curl http://localhost
```

```
# curl: (7) Failed to connect to localhost port 80 after 1 ms: Connection refused
```

```
alpine:~/nginx$ curl http://localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

---

### Lab 3: Host & None Network

```
curl http://localhost
```

```
# Returns nginx default HTML page
```

```
docker run -d --name isolated-app --network none alpine sleep 3600
```

```
docker exec isolated-app ifconfig
```

```
# lo    Link encap:Local Loopback
```

```
#      inet addr:127.0.0.1  Mask:255.0.0.0
```

```
#      inet6 addr: ::1/128 Scope:Host
```

```

anirudh_chaitanya@EC2-5K:~/flask-env-app$ curl http://localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
anirudh_chaitanya@EC2-5K:~/flask-env-app$ docker run -d --name isolated-app --network none alpine sleep 3600

```

# ifconfig continued:

# UP LOOPBACK RUNNING MTU:65536 Metric:1

# RX packets:0 errors:0 TX packets:0 errors:0

# Only loopback - no external network access

docker network create app-network

docker network connect app-network web1

docker network disconnect app-network web1

docker network rm app-network

docker network prune

# Deleted Networks: my\_overlay, my\_ipvlan, 26compose\_wordpress-network, mybridge

## Lab 4: Multi-Container App - Web + Database

docker network create app-network

docker run -d \

--name postgres-db \

--network app-network \

-e POSTGRES\_PASSWORD=secret \

-v pgdata:/var/lib/postgresql/data \

postgres:15

# Pulling postgres:15 - all layers Pull complete

# Status: Downloaded newer image for postgres:15

```

--name postgres-db \
--network app-network \
-e POSTGRES_PASSWORD=secret \
-v pgdata:/var/lib/postgresql/data \
postgres:15
Unable to find image 'postgres:15' locally
15: Pulling from library/postgres
a8a0725a32a6: Pull complete
af2a6e5f285c: Pull complete
b4765ac2e7e6: Pull complete
f7049a93e93f: Pull complete
71a32c7571f4: Pull complete
c45b346571fc: Pull complete
2c38b75ffca2: Pull complete
5ca81a59b0a4: Pull complete
9328c154beae: Pull complete
00f33d580dc3: Pull complete
8ecd190da6: Pull complete
af0c3ea2fe5e: Pull complete
5c152148754e: Pull complete
b9015a4f6b87: Download complete
cebcbab07837b: Download complete
Digest: sha256:31224c609d6ea75ee9522e0f4d02eb202e74754adea4294fdefe9f93c875b657
Status: Downloaded newer image for postgres:15
d48c85ea9f88f4b38dc246d31fdd03e255e301cedb15813640704d52f2ea84ec
```

docker run -d \

--name web-app \

--network app-network \

-p 8080:3000 \

-e DATABASE\_URL="postgres://postgres:secret@postgres-db:5432/mydb" \

-e DATABASE\_HOST="postgres-db" \

node-app

# Error: pull access denied for node-app (image doesn't exist locally)

docker network inspect bridge

# "Name": "bridge"

# "Driver": "bridge"

# "EnableIPv4": true

# "IPAM": { "Config": [{ "Subnet": "172.17.0.0/16" }] }

```

--name web-app \
--network app-network \
-p 8080:3000 \
-e DATABASE_URL="postgres://postgres:secret@postgres-db:5432/mydb" \
-e DATABASE_HOST="postgres-db" \
node-app
Unable to find image 'node-app:latest' locally
docker: Error response from daemon: pull access denied for node-app, repository does not exist or may require 'docker login'
Run 'docker run --help' for more information
```

```

[
  {
    "Name": "bridge",
    "Id": "04c34ceaedb706081edb3c302f46f3273fe40fa65434f49d8a0d7149ab61bbb",
    "Created": "2026-04-09T13:07:53.676393869Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "172.17.0.0/16",
          "Gateway": "172.17.0.1"
        }
      ]
    }
  }
]

```

# bridge inspect continued:

# IPv4Address: "172.17.0.8/16"

docker stop \$(docker ps -q)

docker rm \$(docker ps -aq)

# All container IDs listed and removed

```

d48c85ea9f88
docker rm $(docker ps -aq)3b42026e3e06
bcc806882b97
de979df15bdc
e9a9ab9a3abe
8521cbe18941
45c915b7dacb
d4252783d510
9dd09592720c
d728b47aab0f

```

```

d48c85ea9f88
3b42026e3e06
bcc806882b97
de979df15bdc
e9a9ab9a3abe
8521cbe18941
45c915b7dacb
d4252783d510
9dd09592720c
d728b47aab0f
ecbd2981748c
4afd69c3b1ab

```

---

## Lab 5: Port Publishing

docker ps

# (empty)

docker run -d -p 80:8080 --name app1 nginx

docker run -d -p 8080 --name app2 nginx

docker ps

# app2: 0.0.0.0:56066->8080/tcp (random port assigned)

```
# app1: 0.0.0.0:80->8080/tcp
docker run -d -p 8082:80 -p 8443:443 --name app3 nginx
docker run -d -p 127.0.0.1:8085:80 --name app4 nginx
docker network create myapp-network
docker network ls

#NETWORK ID      NAME      DRIVER  SCOPE
#bb849a0c6540    app-network  bridge  local
#04c34ceeaedb    bridge     bridge  local
#032eba416aaf    host       host    local
#46b5dc6ed097    my-network  bridge  local
#5ed511317523    myapp-network bridge  local
#609c188dce0e    none       null    local
```

```
anirudh_chawla@LUCIF3R:~/flask-env-app$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS    PORTS    NAMES
anirudh_chawla@LUCIF3R:~/flask-env-app$ docker run -d -p 80:8080 --name app1 nginx
216d15b176b733b076e487fcd44e7e24a0576d9c40c8166175fed55e2f7bf0fb
anirudh_chawla@LUCIF3R:~/flask-env-app$ docker run -d -p 8080 --name app2 nginx
8d603b7e14785e52938409b5f80b88f1e7765b47be427242e61aeaf6bd21de84
anirudh_chawla@LUCIF3R:~/flask-env-app$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS    PORTS    NAMES
8d603b7e1478   nginx     "/docker-entrypoint..." 4 seconds ago Up 3 seconds 80/tcp, 0.0.0.0:32768->8080/tcp, [::]:32768->8080/tcp  app2
216d15b176b7   nginx     "/docker-entrypoint..." 9 seconds ago Up 8 seconds 80/tcp, 0.0.0.0:80->8080/tcp, [::]:80->8080/tcp  app1
anirudh_chawla@LUCIF3R:~/flask-env-app$ docker run -d -p 8082:80 -p 8443:443 --name app3 nginx
d8648c4982dbad4ce496baae2a61641b62dc456d48277e7d93bc0b5f7d8256
anirudh_chawla@LUCIF3R:~/flask-env-app$ docker run -d -p 127.0.0.1:8085:80 --name app4 nginx
d8fe0179fc5e63bd19761580bbe137b649b91a16747aeb246b9c7b29ba264a02
anirudh_chawla@LUCIF3R:~/flask-env-app$ docker network create myapp-network
5ed51131752394ca52548188ea60d66b12e55960828cbaac8279e4dce4d9d21
anirudh_chawla@LUCIF3R:~/flask-env-app$ docker network ls
NETWORK ID      NAME      DRIVER  SCOPE
bb849a0c6540    app-network  bridge  local
04c34ceeaedb    bridge     bridge  local
032eba416aaf    host       host    local
46b5dc6ed097    my-network  bridge  local
5ed511317523    myapp-network bridge  local
609c188dce0e    none       null    local
```

## Part 5: Complete Real-World Example

### Flask + PostgreSQL + Redis Setup

```
docker run -d \
  --name redis \
  --network myapp-network \
  -v redis-data:/data \
  redis:7-alpine

# Pulling redis:7-alpine - all layers Pull complete

# Status: Downloaded newer image for redis:7-alpine

mkdir app

echo "print('Flask app placeholder')" > app/app.py

echo "SECRET_KEY=supersecretkey" > .env.production
```

```
echo "APP_ENV=production" >> .env.production
```

```
cat .env.production
```

```
# SECRET_KEY=supersecretkey
```

```
# APP_ENV=production
```

```
docker build -t flask-app:latest .
```

```
# [+] Building 61.0s (7/7) FINISHED
```

```

--name=redis \
--network myapp-network \
-v redis-data:/data \
redis:7-alpine
Unable to find image 'redis:7-alpine' locally
7-alpine: Pulling from library/redis
9fe3744a2eac: Pull complete
1c2c8d1ee428: Pull complete
09a5a8e32a23: Pull complete
bc1da858f299: Pull complete
2a97533d89ca: Pull complete
bd53938b1271: Pull complete
7ad59b3ccef: Pull complete
4f4fb788e54: Pull complete
9bbcfa5d5580: Download complete
06cd2b88db40: Download complete
Digest: sha256:0881dd37f49270ec4e516d41acfb9fe2460070dc6d4a4578a2ac5b9d59df065
Status: Downloaded newer image for redis:7-alpine
dbaf2bba62510de9d53a34550fe8b7374e5e923d77565e0db19e0f9493ec5c
anirudh_chamla@UC1F3R:~/flask-env-app$ mkdir app
anirudh_chamla@UC1F3R:~/flask-env-app$ echo "print('Flask app placeholder!')" > app/app.py
anirudh_chamla@UC1F3R:~/flask-env-app$ echo "SECRET_KEY=supersecretkey" > .env.production
echo "APP_ENV=production" >> .env.production
anirudh_chamla@UC1F3R:~/flask-env-app$ cat .env.production
SECRET_KEY=supersecretkey
APP_ENV=production
anirudh_chamla@UC1F3R:~/flask-env-app$ docker build -t flask-app:latest .
[+] Building 14.7s (9/9) FINISHED
==> [internal] load build definition from Dockerfile
==> == transferring Dockerfile: 300B 0.0s
==> [internal] load metadata for docker.io/library/python:3.9 1.8s
==> [internal] load .dockerignore 0.0s
==> == transferring context: 2B 0.0s
==> [1/9] FROM docker.io/library/python:3.9@sha256:da5aee29682d12a6049f31c046f15b070eb3edc5200923c01d0cb330ed87c913 0.0s
==> == resolve docker.io/library/python:3.9@sha256:da5aee29682d12a6049f31c046f15b070eb3edc5200923c01d0cb330ed87c913 0.0s
==> [internal] load build context 0.0s
==> == transferring context: 796B 0.0s
==> CACHED [2/9] WORKDIR /app 0.0s
==> [3/9] COPY . 0.0s
==> [4/9] RUN pip install flask 12.2s
==> exporting to image 0.0s
==> == exporting layers 0.0s
==> == exporting manifest sha256:f0070b109324c3502891c80cc05aac4e4fa7dbf0d5e0bf704f08ac20e2164c990 0.0s
==> == exporting config sha256:30302fae0ff40c90e08e0d50f40da9e304101d4140140c70a19331a0537707 0.0s
==> == exporting attestation manifest sha256:f28a37bc3d1c1e509ff703063a50b06f70a7533050273c1d2f774de90972b 0.0s
==> == exporting manifest list sha256:09f61caa51404fb5a9260930f30cc2ba3307a0940a04a0d100f5f040bbee29 0.0s
==> == naming to docker.io/library/flask-app:latest 0.0s
==> == unpacking to docker.io/library/flask-app:latest 0.1s
docker:default
```

## Monitoring All Services

```
# docker stats (no-stream) - postgres, redis, flask-app all showing --
```

```
# Ctrl+C
```

```
docker ps
```

| #CONTAINER ID                                                                      | IMAGE          | COMMAND                  | CREATED            | STATUS | PORTS          |
|------------------------------------------------------------------------------------|----------------|--------------------------|--------------------|--------|----------------|
| NAMES                                                                              |                |                          |                    |        |                |
| #dbaf2bba6251                                                                      | redis:7-alpine | "docker-entrypoint.s..." | About a minute ago | Up     | About a minute |
| 6379/tcp                                                                           |                | redis                    |                    |        |                |
| #d8fe0179fc5e                                                                      | nginx          | "/docker-entrypoint...." | 3 minutes ago      | Up     | 3 minutes      |
| 127.0.0.1:8085->80/tcp                                                             |                | app4                     |                    |        |                |
| #d8648c4982db                                                                      | nginx          | "/docker-entrypoint...." | 3 minutes ago      | Up     | 3 minutes      |
| 0.0.0.0:8082->80/tcp, [::]:8082->80/tcp, 0.0.0.0:8443->443/tcp, [::]:8443->443/tcp |                | app3                     |                    |        |                |
| #8d603b7e1478                                                                      | nginx          | "/docker-entrypoint...." | 3 minutes ago      | Up     | 3 minutes      |
| 0.0.0.0:32768->8080/tcp, [::]:32768->8080/tcp                                      |                | app2                     |                    |        | 80/tcp,        |
| #216d15b176b7                                                                      | nginx          | "/docker-entrypoint...." | 3 minutes ago      | Up     | 3 minutes      |
| 0.0.0.0:80->8080/tcp, [::]:80->8080/tcp                                            |                |                          |                    |        | 80/tcp,        |

```
root@b8c1b2c0c2d2: / # docker ps
```

| CONTAINER ID | IMAGE          | COMMAND                   | CREATED            | STATUS            | PORTS                                                                              | NAMES |
|--------------|----------------|---------------------------|--------------------|-------------------|------------------------------------------------------------------------------------|-------|
| dbaf2bba6251 | redis:7-alpine | "/docker-entrypoint.s..." | About a minute ago | Up About a minute | 6379/tcp                                                                           | redis |
| d8fe0179fc5e | nginx          | "/docker-entrypoint..."   | 3 minutes ago      | Up 3 minutes      | 127.0.0.1:8085->80/tcp                                                             | app4  |
| d6d48c4982db | nginx          | "/docker-entrypoint..."   | 3 minutes ago      | Up 3 minutes      | 0.0.0.0:8082->80/tcp, [::]:8082->80/tcp, 0.0.0.0:8443->443/tcp, [::]:8443->443/tcp | app3  |
| 8d683b7e1478 | nginx          | "/docker-entrypoint..."   | 3 minutes ago      | Up 3 minutes      | 80/tcp, 0.0.0.0:32768->8080/tcp, [::]:32768->8080/tcp                              | app2  |
| 216d15b176b7 | nginx          | "/docker-entrypoint..."   | 3 minutes ago      | Up 3 minutes      | 80/tcp, 0.0.0.0:80->8080/tcp, [::]:80->8080/tcp                                    | app1  |

## Key Takeaways

1. **Volumes** persist data beyond container lifecycle
2. **Environment variables** configure containers dynamically
3. **Monitoring commands** help debug and optimize containers
4. **Networks** enable secure container communication
5. **Always use named volumes** for production data
6. **Custom networks** provide better isolation and DNS
7. **Monitor resource usage** to prevent issues
8. **Use .env files** for sensitive configuration